



Detecting hallucinations with LLM-as-a-judge: Prompt engineering and beyond

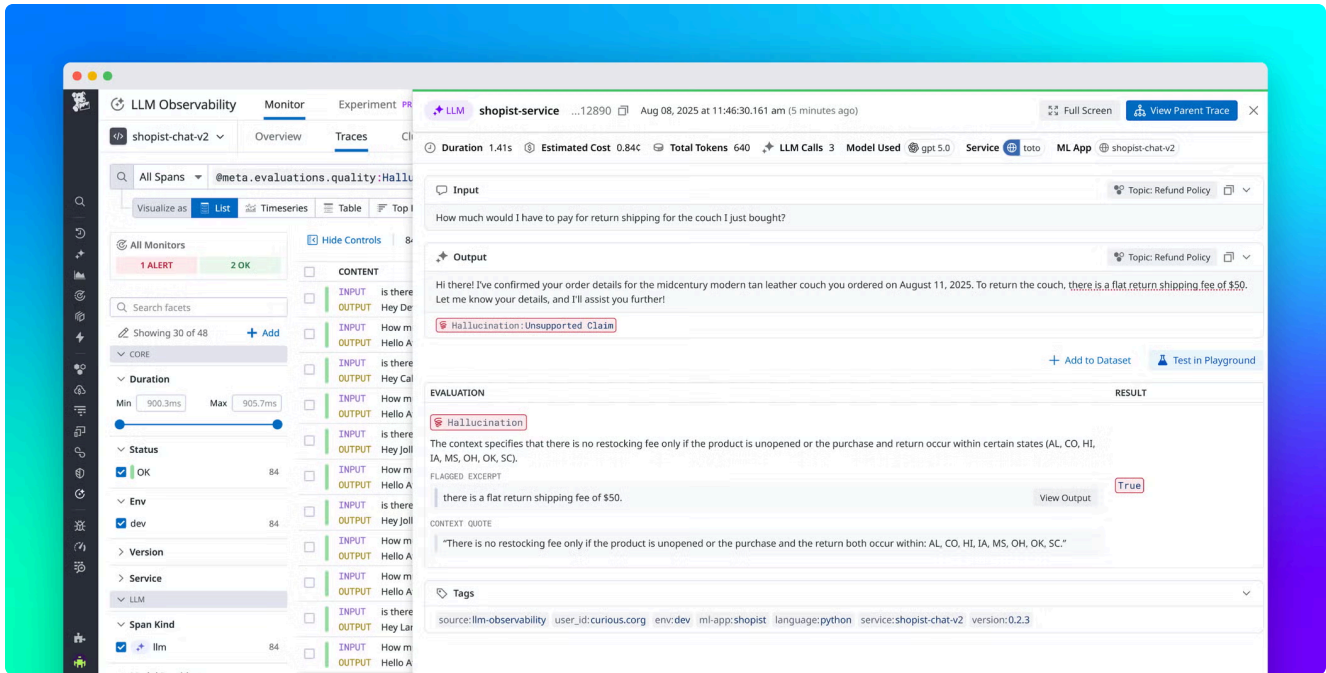
PUBLISHED

Aug 25, 2025

READ TIME

10m

LLM OBSERVABILITY AI MACHINE LEARNING AI RESEARCH





detection methods

Beyond prompt engineering

Results

Limited results

What's next

References

high-profile incidents that suggest even these deeply engaging energy hallucinations remain a persistent issue in recent model releases ^{1,2}.

Datadog LLM Observability monitors LLM-based applications in production. This naturally leads us to tackle the challenge of detecting hallucinations in real time. Flagging these critical failures in production allows our customers to assess their LLM applications' performance, reach out to affected end users, and iterate on their LLM applications to build a more reliable system.

In this post, we'll explore how Datadog built a **hallucination detection feature** for use cases like retrieval-augmented generation (RAG) ³. We'll review various approaches to detect hallucinations, and explain our focus on LLM-as-a-judge approaches with optimized judgment prompts. We'll share some of our encouraging results and discuss future R&D directions for detecting hallucinations.

Augmented generation: Context is everything

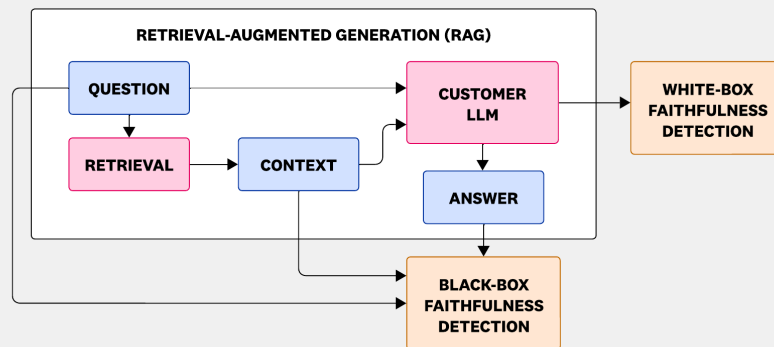
Hallucination is an umbrella term used to describe various distinct undesirable LLM behaviors. This post will focus on **faithfulness**, a specific requirement for non-hallucinated answers in use cases involving RAG. Determining whether an answer is faithful involves three required components:

- **A question** posed by an end user
- **Context** containing information required to answer the question, usually retrieved from an accepted knowledge base using RAG or similar techniques
- **An answer** generated by the LLM

Faithfulness is the requirement that an LLM-generated answer agrees with the context. Crucially, the question of faithfulness assumes that the context is correct and accepts it as ground truth; verifying the context is an independent problem.

Faithfulness does not apply to use cases without a context to use as the ground truth. In such cases, hallucination detection must rely on other methods like confabulation detection or white-box techniques. Datadog focuses on faithfulness since augmented generation techniques, which always retrieve a context, remain the tool of choice to adapt general-purpose models to specific domains.

How we evaluate faithfulness in retrieval-augmented generation using white- and black-box methods



Retrieval-augmented generation (RAG) begins with a question from the end user. A retrieval process fetches relevant context needed to answer the question, and then the LLM is prompted to provide an answer based on this context. Faithfulness asks whether this answer agrees with the context. White-box faithfulness detection methods inspect the LLM directly, whereas black-box methods can be applied without access to the underlying LLM's internal weights.

LLM-as-a-judge vs. alternative detection methods

Multiple techniques have been developed for detecting hallucinations in general, and unfaithful answers specifically.



In contrast, **black-box** detection methods can only see the input to and output from the generating LLM. **Perturbation-based** approaches measure the reproducibility of the answer by regenerating it under different conditions ^{7 8}. **SLM-as-a-judge** approaches use small language models (e.g., BERT-style) to judge answer correctness ⁹. **LLM-as-a-judge** approaches use a distinct judge model to evaluate the correctness of the generator model ^{10 11 12}.

At Datadog, we focus on black-box detection for our LLM Observability product to support the full range of customer use cases, including black-box model providers. Within black-box detection, perturbation-based approaches are often prohibitively expensive as they rely on regenerating the answer from the original LLM, often resulting in a **5-10x** cost increase due to the number of LLM calls.

SLM-as-a-judge approaches are promising, but begin to fail in complex use cases, especially when the context and answer become large and involve layers of reasoning. Given these challenges, we focus on LLM-as-a-judge approaches for monitoring RAG-based apps in production.

Beyond prompt engineering

LLM-as-a-judge methods have two main components: the model and the prompting approach. Most of the literature focuses on creating better models through fine-tuning on curated or synthetically generated data. In contrast, prompting is often overlooked, perhaps because it appears simpler.

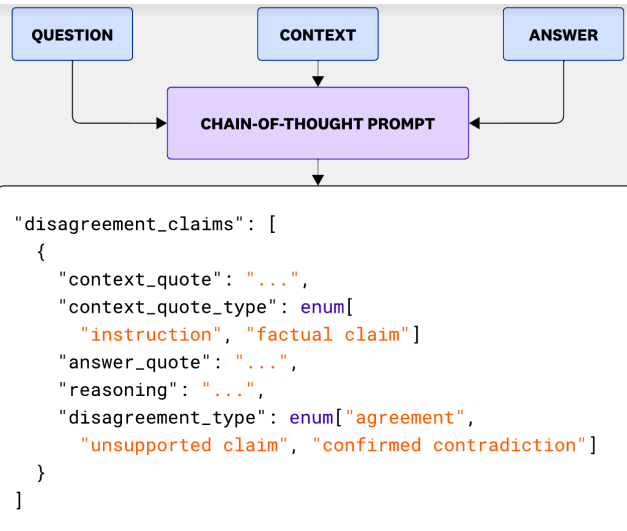
In the world of LLM agents, prompt engineering today involves more than optimizing a single string. Defining a logical flow across multiple LLM calls and using format restrictions like structured output can effectively create guidelines for LLM output. Our results show that a good prompting approach that breaks down the task of detecting hallucinations into clear steps can achieve significant accuracy gains.

We also believe that gains achieved through prompt engineering can transfer to fine-tuned models. Prompts are often used to augment labeled data with reasoning chains for supervised fine-tuning (SFT) or in SFT initialization steps before reinforcement learning (RL).

Creating a rubric for detecting hallucinations

A key principle in our prompt optimization approach is to recognize that **LLMs are better at guided summarization than complex reasoning**. Therefore, if we can break down a problem into multiple smaller steps of guided summarization by creating a **rubric**, we can play to the strengths of LLMs and then rely on simple deterministic code to parse the LLM output and score the rubric.

The following diagram shows our initial prompting approach for detecting hallucinations. This process asks the LLM-as-a-judge to fill out a rubric, with instructions defining specific types of hallucination. A final boolean decision is made through deterministic parsing of the filled-out rubric. Let's examine the rubric in detail.



Our initial, single-step prompting approach to detect hallucinations with LLM-as-a-judge.

The rubric for hallucinations is a list of **disagreement claims**. The task is framed as *finding all claims where the context and answer disagree*. Agreement can only be determined by exhaustively searching for possible disagreements and finding none.

For each claim, the rubric requires a quote from the context and the answer, showing the relevant portions of each text. Repeating these quotes forces the LLM-as-a-judge generation to remain grounded in the text. We also highlight these quotes in our UI, making it clear to customers which part of a long context may have been misinterpreted or contradicted by the LLM, and which part of a long answer contains the hallucination.

We classify the type of context quote, and most importantly, the type of disagreement. These offer customizable options for our customers. In particular, we distinguish between two types of disagreements:

- **Contradictions** are claims made in the LLM-generated answer that go directly against the provided context. Since we assume the context is correct, these claims are likely to be strictly incorrect.
- **Unsupported claims** are parts of the LLM-generated answer that are not grounded in the context. These claims may be based on other knowledge outside the context, or they may be a result of jumping to conclusions beyond the specific scope in the context.

In sensitive use cases, LLM's are often expected to respond only based on the provided context —without relying on external information or making new conclusions. In these situations, customers can select both **contradictions** and **unsupported claims** to be flagged.

In less sensitive use cases, it may be acceptable for an LLM to incorporate outside knowledge or make reasonable assumptions. For these situations, customers could unselect **unsupported claims** and choose to only flag **contradictions**.

The rubric also allows the model to invalidate a disagreement by labeling its type as an **agreement**. Often, the LLM may begin generating tokens for a disagreement, but after repeating the relevant quotes from the context and answer and generating reasoning tokens, determine that this particular claim is actually an agreement. In these cases, the **agreement** option serves as a route to “back out” of a disagreement through reasoning.

Structured output

Since our hallucination rubric requires specific fields to be filled out, we need a way to enforce this format on the LLM's output. We use **structured output** to ensure the LLM generates not only valid JSON, but also adheres to the rubric's format provided.

There are various implementations of structured output and other format restrictions, but the most common method pairs an LLM with a **finite state machine (FSM)**. As the LLM generates tokens, the FSM checks the previously generated tokens and the desired output format to



generated tokens can have undesirable effects on the LLM's performance and accuracy¹³.

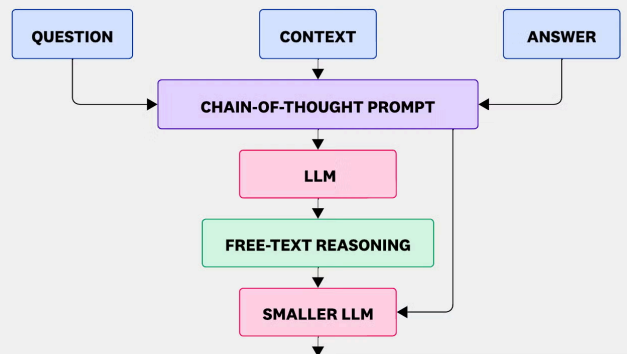
Replicating reasoning behaviors without dedicated reasoning models

Restricting the LLM's output format is a double-edged sword: While structural constraints can guide reasoning by enforcing a consistent format, strict enforcement may hinder the model's ability to reason effectively. Dedicated reasoning models typically address this by generating output in two distinct stages: reasoning generation with minimal restrictions on formatting, then final answer generation.

This two-stage reasoning approach is trained through reinforcement learning. Notably, large-scale reinforcement learning elicits desirable reasoning behaviors like hypothesis generation and self-criticism as emergent properties. We replicate some of these behaviors through explicit two-stage prompting of non-reasoning models—that is, models that were not directly trained with reinforcement learning to produce output in this two-stage format. Eliciting reasoning behavior from non-reasoning models is desirable because dedicated reasoning models are often more expensive to train and can have hidden reasoning tokens.

The following diagram shows our two-step prompting approach.

How we prompt non-reasoning models to emulate two-stage reasoning behavior



```
"disagreement_claims": [
  {
    "context_quote": "...",
    "context_quote_type": enum[
      "instruction", "factual claim"
    ],
    "answer_quote": "...",
    "reasoning": "...",
    "disagreement_type": enum["agreement",
      "unsupported claim", "confirmed contradiction"
    ]
  }
]
```

Our two-step prompting approach to detect hallucinations with LLM-as-a-judge. This approach replicates some of the behaviors of reasoning models, but through explicit prompting rather than as emergent properties of reinforcement learning.

We initially prompt an LLM to fill out our rubric without any restrictions on output format. In this stage, we include instructions in the prompt that encourage self-criticism, consideration of multiple interpretations, and other reasoning behaviors.

We then make a second LLM call using structured output to convert the output to the desired format. As this step involves simple summarization and reformatting, we use a smaller LLM for it to save resources.

Word choice also affects the prompt's effectiveness. We rename the context and answer in our chain-of-thought prompt and in the final rubric, consistently referring to the context as **expert**



There are a variety of open source hallucination benchmarking datasets^{12 14} ranging across topics, using various definitions of hallucination, and using different mechanisms for calibration and verification.

We report results on two benchmarks:

- **HaluBench**¹² is a partially synthetic benchmark. Negative examples (non-hallucinated answers) come from existing question answering (QA) benchmarks like HaluEval, DROP, CovidQA, FinanceBench, and PubMedQA. Positive examples (hallucinated answers) are generated by prompting an LLM to modify the correct answer.
- **RAGTruth**¹⁴ is a fully human-labeled benchmark covering three tasks: QA, summarization, and data-to-text translation. All answers are LLM-generated, ensuring that positive and negative examples come from the same distribution.

We compare our detection method against two baselines: the open source **Lynx (8B)** from **Patronus AI**, and the same prompt used by **Patronus AI** evaluated on **GPT-4o**. All three detection methods use the same faithfulness evaluation format: question, context, answer.

Limited results

PROMPT AND MODEL ↓	PRECISION	RECALL	F1	PRECISION	RECALL	F1
Datadog approach: GPT-4o	0.788	0.833	0.810	0.788	0.833	0.810
Patronus prompt: Lynx (8B)	0.637	0.861	0.733	0.637	0.861	0.733
Patronus prompt: GPT-4o	0.905	0.681	<u>0.777</u>	0.905	0.681	<u>0.777</u>

For all three detection methods tested, F1 scores are consistently higher on HaluBench than on RAGTruth, suggesting that RAGTruth is a more difficult benchmark. This result is not surprising given that RAGTruth samples positive and negative examples from the same distribution of LLM-generated responses.

Notably, the Datadog detection method shows the smallest drop in F1 between HaluBench and RAGTruth. This suggests that our detection remains robust as the hallucinations become harder to detect.

What's next

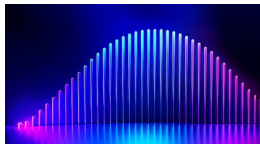
Our work on LLM-as-a-judge shows that prompt design—not just model architecture—can significantly improve hallucination detection in RAG-based applications. By enforcing structured output and guiding reasoning through explicit prompts, we've built a robust approach that performs well even on challenging human-labeled benchmarks.

In the coming months, we hope to continue refining our internal benchmarks and share them. This data can then be used to fine-tune an LLM-as-a-judge, combining our efforts in prompt engineering and fine-tuning to achieve a best-in-class hallucination detection mechanism.

If this kind of work intrigues you, consider applying to work at Datadog! [We're hiring!](#)

FURTHER READING

Forrester Wave™: AIOps Platforms, Q2 2025



Learn why Datadog is named a Leader in The Forrester Wave™: AIOps Platforms, Q2 2025.

[GET THE REPORT](#)

References

1. [Walk the Talk? Measuring the Faithfulness of Large Language Model Explanations](#) ↗
 2. [AI Hallucination: Comparison of the Popular LLMs in 2025](#) ↗
 3. [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#) ↗
 4. Ji, Z. et al. (2023). Detecting hallucinations in large language models using semantic entropy. *Nature*. ↗
- [Detecting hallucinations in Large Language Model Generation: A Token Probability Approach](#) ↗

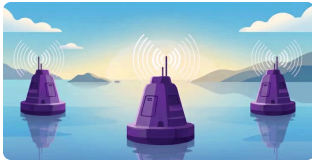


9. [SemScore: Automated Evaluation of Instruction-Tuned LLMs Based on Semantic Textual Similarity](#)
10. [Liu, X. et al. \(2023\). G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. ACL 2023.](#)
11. [ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems](#)
12. [Lynx: An Open Source Hallucination Evaluation Model](#)

Articles



Toto 2.0: Time series forecasting enters the scaling era



Monitoring LangGraph agents with Datadog: a practical guide



How we made a SQL query optimization agent 59% more accurate using autoresearch and...



Introducing ARFBench: A time series question-answering benchmark based on real incidents

Related jobs at Datadog

We're always looking for talented people to collaborate with

FEATURED POSITIONS

Director, Product Management - AI Observability

PRODUCT MANAGEMENT

[VIEW ALL](#)

Start monitoring your metrics in minutes

[FIND OUT HOW](#)

FREE TRIAL

Download mobile app



PRODUCT

- Infrastructure Monitoring
- Network Monitoring
- Container Monitoring
- Serverless
- Cloud Cost Management
- Cloudcraft
- Kubernetes Autoscaling
- Application Performance Monitoring
- Software Catalog
- Universal Service Monitoring
- Data Streams Monitoring
- Jobs Monitoring
- Quality Monitoring
- Database Monitoring
- Continuous Profiler
- Dynamic Instrumentation
- Log Management
- Sensitive Data Scanner
- BYOC Log Management
- Internal Developer Portal
- CI Visibility
- Test Optimization
- Feature Flags
- Code Coverage
- Service Level Objectives
- Incident Response
- Event Management
- Case Management
- Bits AI Agents
- Bits AI SRE
- Bits AI Security Analyst
- MCP Server
- Pup CLI
- Agent Directory
- Metrics
- Watchdog

RESOURCES

- Pricing
- Documentation
- Support
- Services & Enablement
- Product Preview Program
- Certification
- Open Source
- Events and Webinars
- Security
- Privacy Center
- Knowledge Center
- Learning Resources

ABOUT

- Contact Us
- Partners
- Press
- Leadership
- Careers
- Legal
- Investor Relations
- Analyst Reports
- ESG Report
- Vendor Help
- Trust Hub

BLOG

- The Monitor
- Engineering
- AI
- Security Labs



- Entitlement Management
- Vulnerability Management
- Compliance
- App and API Protection
- Software Composition Analysis
- Code Security
- Static Code Analysis (SAST)
- Runtime Code Analysis (IAST)
- IaC Security
- Cloud SIEM
- Browser Real User Monitoring
- Mobile Real User Monitoring
- Product Analytics
- Experiments
- Session Replay
- Synthetic Monitoring
- Mobile App Testing
- Continuous Testing
- Error Tracking
- Secrets
- Notebooks
- Mobile App
- Fleet Automation
- Governance Console
- Access Control
- OpenTelemetry
- Alerts
- Integrations
- IDE Plugins
- API
- Marketplace
- Security Labs Research
- Open Source Projects
- Storage Management
- GPU Monitoring
- DORA Metrics
- Secret Scanning